Estação de Campo e Central de Irrigação

Projeto 3 · Comunicação serial UART cliente–servidor · Sistemas Embarcados — UPE
Giulia Buonafina · Maria Luana Rodrigues · NUCLEO-L476RG (cliente) + STM32F407 Discovery (servidor) · STM32CubeIDE / HAL

1. Contexto do problema

A agricultura é a atividade que mais consome água doce no mundo, e grande parte desse consumo vem da irrigação. Em muitas propriedades irriga-se por horário fixo, **sem saber quanto choveu** em cada talhão. Isso causa desperdício de água e de energia, excesso de umidade e deslocamentos até o campo apenas para ler o pluviômetro.

2. Solução proposta

Dois nós ligados por **UART**, em arquitetura cliente–servidor:

- **Estação de Campo — servidor (Discovery):** conta as basculadas do pluviômetro (cada uma = 0,25 mm de chuva; simulada pelo botão azul) e guarda o cadastro dos talhões.
- **Central de Irrigação — cliente (Nucleo):** quando o operador aperta o botão B1, pede os dados à estação, sinaliza com o LED (1 piscada por basculada, a 1 Hz), decide se a irrigação deve ser **suspensa** (chuva ≥ 2,00 mm) e envia o relatório ao computador.

115200	**0x5A**	**0,25 mm**	**2,00 mm**
baud, formato 8N1	byte de pedido	chuva por basculada	limite (8 basculadas)

3. Dados transmitidos e protocolo

#	Mensagem	Direção	Tam.	Conteúdo (significado no problema)	Mecanismo
①	Comando de coleta	Nucleo → Discovery	1 B	`0x5A` — "envie a chuva acumulada"	Interrupção
②	Contador de basculadas	Discovery → Nucleo	1 B	número de 0 a 255 (× 0,25 mm = chuva)	Interrupção
③	Tabela de talhões	Discovery → Nucleo	100 B	texto ASCII: "Talhao 1, Soja, 10 mm" …	DMA
④	Relatório	Nucleo → PC	~55 B	"Basculadas = 003 | Chuva = 0.75 mm | Irrigacao: LIBERADA"	Interrupção
⑤	Tabela repassada	Nucleo → PC	100 B	texto ASCII (mesma tabela)	DMA

O servidor nunca fala sozinho: só responde ao `0x5A`. A ordem é fixa (contador, depois tabela) e a tabela tem tamanho fixo de 100 bytes, para o DMA saber quantos bytes esperar. Se a estação não responder em 2 s, a central cancela a coleta e avisa o PC.

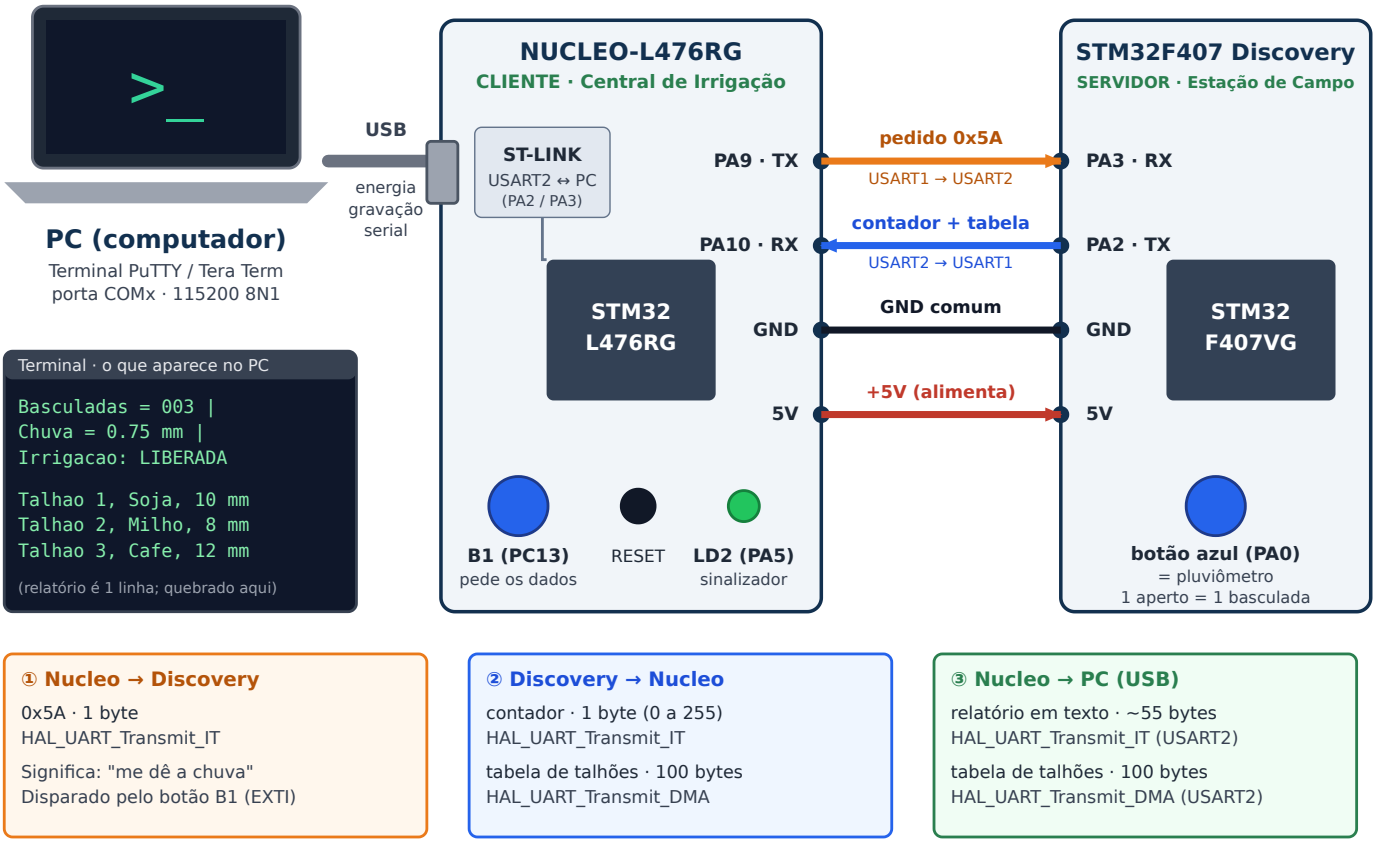
4. Configuração da UART e dos periféricos

UART	Valor
Baud rate	**115200**
Dados / paridade / stop	**8 / None / 1** (8N1)
Modo / fluxo	TX e RX / sem controle
Nível lógico	3,3 V
Tempo	1 byte ≈ 87 µs · 100 bytes ≈ 8,7 ms

115200 8N1: padrão do CubeMX; os dois lados devem ser idênticos. `0x5A`: comando da Lista 3.

Item	Configuração
B1 Nucleo (PC13)	EXTI, borda de descida, pull-up
Botão Discovery (PA0)	EXTI, subida e descida, filtro de 100 ms
LED (PA5)	saída push-pull
NVIC	EXTI e USART habilitados
DMA	USART1_RX e USART2_TX (Nucleo); USART2_TX (Discovery); modo Normal

5. Montagem física e pinagem



Um único cabo USB na Nucleo: alimenta, grava e leva a serial ao PC. A Discovery é alimentada pela Nucleo (fio 5V + GND comum).

Figura 1 — Ligação da bancada: Nucleo = cliente (USART1) e Discovery = servidor (USART2). Um único cabo USB, na Nucleo, alimenta, grava e leva a serial ao PC; a Discovery recebe 5 V e GND pelos fios.

Ligações entre as placas

Nucleo	Discovery	Função
PA9 (TX)	PA3 (RX)	dados →
PA10 (RX)	PA2 (TX)	← dados
GND	GND	referência comum
5V	5V	alimentação

Pinagem

	Nucleo	Discovery
Botão	PC13 (EXTI)	PA0 (EXTI)
LED	PA5 (LD2)	—
UART placas	USART1 PA9/PA10	USART2 PA2/PA3
UART PC	USART2 PA2/PA3 (USB)	—

6. Sequência de uma coleta

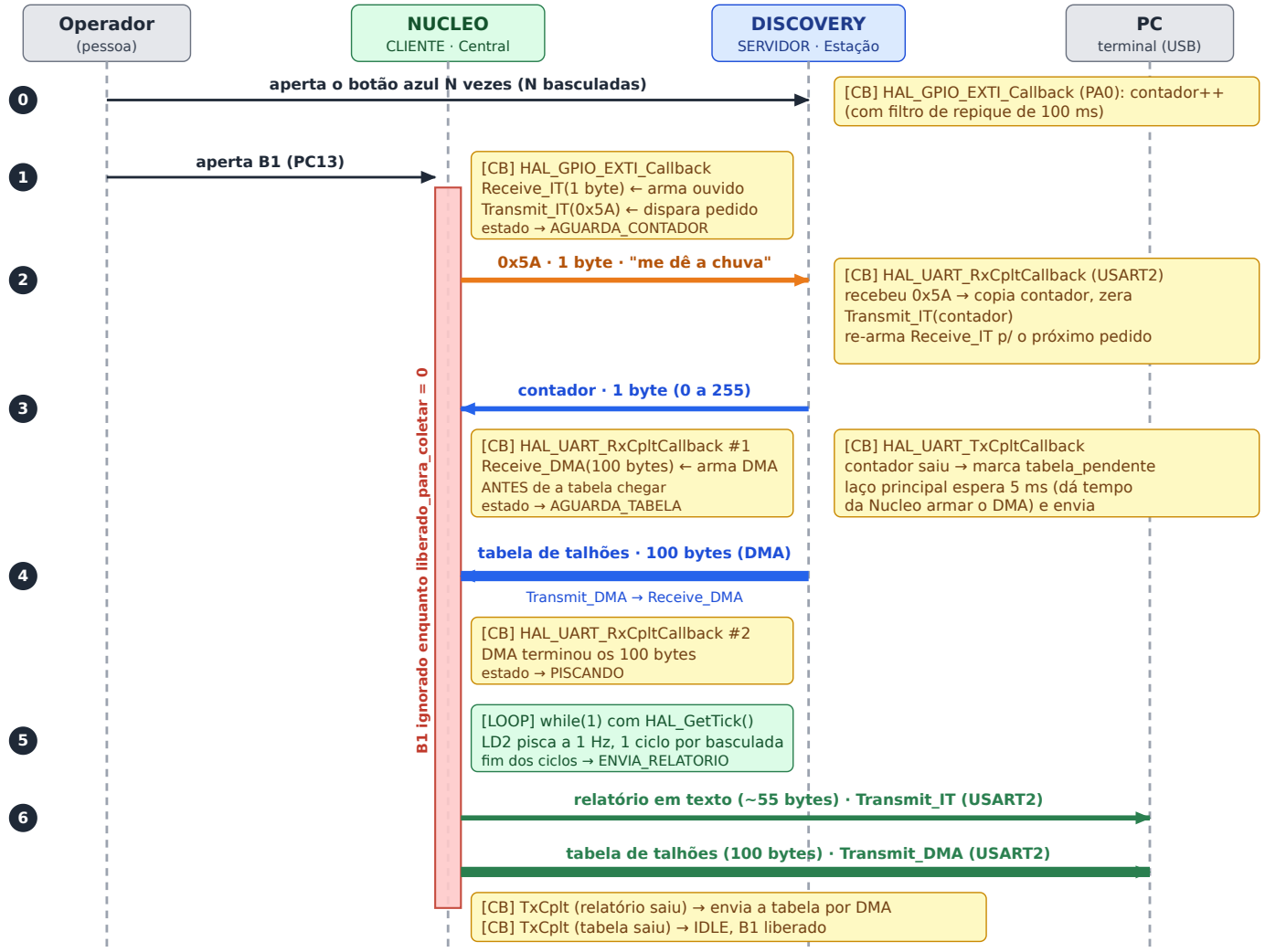


Figura 2 — Caixas amarelas = código executado em callbacks (disparados por interrupção). Barra vermelha = período em que o B1 é ignorado.

7. Polling, interrupção e DMA: onde usamos e por quê

Mecanismo	Como funciona	Onde foi usado	Por quê
Polling	a CPU espera ou pergunta até terminar	`HAL_UART_Transmit` (bloqueante) no banner inicial; `HAL_GetTick()` no laço (piscar, timeout, pausa de 5 ms)	banner roda uma vez; `HAL_GetTick` mede tempo sem travar a CPU como o `HAL_Delay`
Interrupção (EXTI)	o hardware avisa; a CPU executa o callback e volta	B1 da Nucleo (inicia a coleta) e botão azul da Discovery (conta basculada)	apertos imprevisíveis e curtos, não podem ser perdidos
Interrupção (UART `_IT`)	inicia e retorna na hora; avisa por callback	`Transmit_IT` / `Receive_IT`: comando, contador e relatório	mensagens curtas: DMA seria exagero
DMA	o controlador move os bytes sozinho; avisa no fim	`Receive_DMA` da tabela (Nucleo); `Transmit_DMA` da tabela (Discovery e Nucleo → PC)	100 bytes ≈ 8,7 ms com a CPU livre

Callback: função chamada pelo HAL quando algo que pedimos terminou (`HAL_GPIO_EXTI_Callback`, `HAL_UART_RxCpltCallback`, `HAL_UART_TxCpltCallback`). As funções `_IT` e `_DMA` retornam imediatamente; o callback dá o próximo passo. **Desvantagem do polling:** enquanto espera, a CPU não faz mais nada e pode perder eventos.

8. Máquina de estados do cliente (Nucleo)

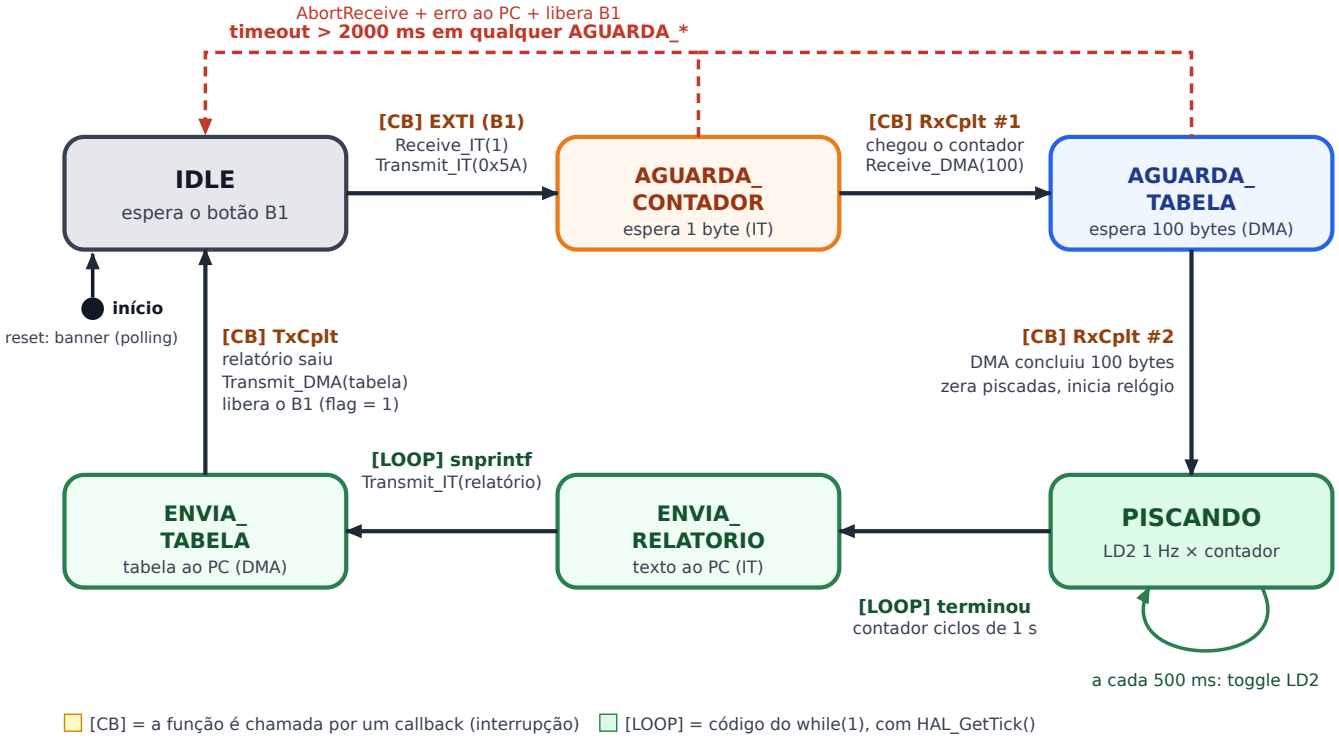
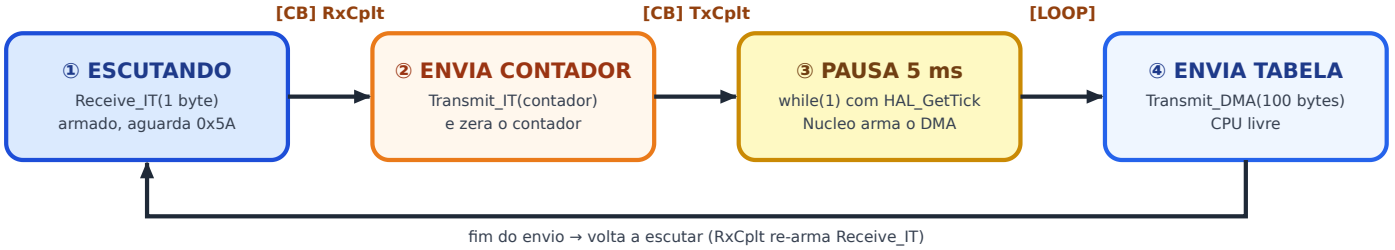


Figura 3 — [CB] = dentro de um callback; [LOOP] = no while(1). Enquanto a coleta não termina, a flag `liberado_para_coletar` (volatile) faz o B1 ser ignorado.

Fluxo passo a passo

1. **IDLE (estado inicial).** Após o reset, o `main` envia um banner ao PC com `HAL_UART_Transmit` (polling) e fica no `while(1)`.
2. **Apertamos o B1.** A EXTI chama `HAL_GPIO_EXTI_Callback`: bloqueia novos cliques, guarda a hora (`HAL_GetTick`), vai para **AGUARDA_CONTADOR**, arma `HAL_UART_Receive_IT` (1 byte) e envia `0x5A` com `HAL_UART_Transmit_IT`.
3. **Chegou o contador.** O HAL chama `HAL_UART_RxCpltCallback`: vai para **AGUARDA_TABELA** e chama `HAL_UART_Receive_DMA` (100 bytes) — antes de a tabela chegar, para o DMA estar pronto (o servidor espera 5 ms).
4. **Chegou a tabela.** O DMA preenche os 100 bytes sem a CPU; ao terminar, `RxCpltCallback` é chamado de novo, zera as piscadas e vai para **PISCANDO**.
5. **PISCANDO.** No `while(1)`, a cada 500 ms o LD2 inverte; cada ciclo de 1 s é uma piscada. Ao completar o valor do contador, vai para **ENVIA_RELATORIO** (contador 0: não pisca).
6. **ENVIA_RELATORIO.** Monta o texto com `snprintf` (chuva = contador × 0,25 mm; SUSPENSA se ≥ 2,00 mm), envia com `Transmit_IT` (USART2) e vai para **ENVIA_TABELA**.
7. **Relatório enviado.** `HAL_UART_TxCpltCallback` dispara `Transmit_DMA` da tabela, volta a **IDLE** e libera o B1.
8. **Falha.** Mais de 2000 ms em AGUARDA_*: `HAL_UART_AbortReceive`, erro ao PC, volta a IDLE e libera o B1.

9. Servidor (Discovery)



Sempre ativo, em paralelo: EXTI do botão azul (PA0)
[CB] HAL_GPIO_EXTI_Callback → se passaram ≥ 100 ms do último evento e o pino está em nível ALTO (apertou), contador++ (máx. 255).
Disparo nas duas bordas (subida e descida); toda borda reinicia a janela de 100 ms — é o filtro de repique do botão.

Figura 4 — O servidor só reage; a contagem de basculadas nunca para.

Ao ligar, o servidor monta a tabela de talhões e arma `Receive_IT` de 1 byte. Ao chegar `0x5A`, `RxCpltCallback` copia o contador, **zera** o contador, envia-o com `Transmit_IT` e re-arma a recepção. Ao fim, `TxCpltCallback` marca "tabela pendente"; o `while(1)` espera 5 ms e chama `Transmit_DMA`. A contagem é independente: cada aperto do botão azul gera uma EXTI que soma ao contador se passaram ≥ 100 ms desde o último evento e o pino está em nível alto (filtro de repique), com saturação em 255.

10. Principais funções

Função	Papel
`HAL_UART_Transmit`	envia e **espera** terminar (polling): banner inicial
`HAL_UART_Transmit_IT` / `Receive_IT`	iniciam e retornam na hora; avisam por callback: comando, contador, relatório
`HAL_UART_Transmit_DMA` / `Receive_DMA`	o DMA move os bytes: tabela de 100 bytes (retornam `HAL_BUSY` se a UART estiver ocupada)
`HAL_GPIO_EXTI_Callback`	chamada na borda do pino configurado (B1 e botão azul)
`HAL_UART_RxCpltCallback` / `TxCpltCallback`	recepção/transmissão completa; avançam a máquina de estados e encadeiam os envios
`HAL_UART_AbortReceive`	cancela uma recepção pendente (timeout de 2 s)
`HAL_GetTick`	milissegundos desde o boot: temporização sem bloqueio

11. Demonstração

Terminal serial no PC (porta COM da Nucleo, 115200 8N1). Aperta-se o botão azul da Discovery N vezes (N basculadas) e, em seguida, o B1 da Nucleo. Comportamento esperado:

Basculadas	LED	Relatório no PC	B1 durante o piscar
0	não pisca	Chuva = 0.00 mm · LIBERADA	—
3	3 piscadas	Chuva = 0.75 mm · LIBERADA	ignorado
8	8 piscadas	Chuva = 2.00 mm · SUSPENSA	ignorado